

Spec-driven development

Unpacking one of 2025's key new AI-assisted engineering practices

Agile engineering practices

Generative AI

Blog

By Liu Shangqi

Published: December 04, 2025

Spec-driven development may not have the visibility of a term like vibe coding, but it's nevertheless one of the most important practices to emerge in 2025. It also highlights how quickly software engineering has evolved in recent months as it has adapted to new AI tools and found new ways of leveraging them.

But as with any new term or practice, understanding exactly what it is is difficult. And there are also questions about how effective a technique where everything is seemingly defined up front really is: can we really consider it agile? In this blog post I'll dive into my take on what it is and discuss how it's being done.

Defining spec-driven development and competing interpretations of it

My understanding of spec-driven development (SDD) is that it's a development paradigm that uses well-crafted software requirement specifications as prompts, aided by AI coding agents, to generate executable code.

It's worth noting though that there are different opinions within the industry about what a spec is and its role in SDD. At the more radical end of the spectrum, there's an argument that we can now discard code and treat specs as the sole source of truth that needs maintenance. In this view, code is a kind of byproduct, an intermediate product between requirements and compiled binaries. In contrast, more old-school technologists — like me — believe specs are merely elements that drive code generation, as it does in test-driven development. Executable code remains the source of truth you need to maintain.

Part of the reason for this disagreement is how approaches have evolved over the last two years since the industry began using generative AI to produce code. When we were first exploring serious programming using ChatGPT, we found that code generated by setting technical specifications (technology stack, architectural style, coding style) using chain-of-thought and few shot prompting greatly was of higher quality compared to plain-text requirements. More recently, though, as AI coding tools have improved, it has become easier to bring functional requirements into the picture.

While this clearly has benefits, we can't rely on functional requirements alone: we need to pay attention to the technical details.

The context of spec-driven development's emergence

Manipulating computers with natural language that represents business has always been the holy grail of software development and programming language theory. In fact, attempts at spec-driven code generation pre-date the LLM era — they've just never reached the level of actual development.

Specs have been used in a number of different ways in software engineering. In distributed computing and RPC communication, for example, specs act as communication contracts between different heterogeneous systems. This usually involves a lot of tedious cross-cutting layer work, including cross-language data type validation, routing, interceptors and observability.

In behavior-driven development (BDD), meanwhile specs are used as a vehicle to facilitate collaboration with business users. They're typically written as scenarios and examples and treated as living documents of system behavior. This is supported by automated testing and continuous integration mechanisms.

However they're used, they're ultimately text-based instructions — and given LLMs ability to manipulate text, it's unsurprising that specs may play so nicely with the growth of AI in software engineering.

Of course, early coding assistants like GitHub Copilot were focused only on generating code snippets. It's only as the context windows of foundation models have grown that it's become easier to write code and build software directly from natural language requirements.

To facilitate this, the latest AI coding agents generally separate the planning and implementation phases of the development process in some form. The planning phase focuses on understanding requirements, designing constraints and better curating prompts for subsequent stages. This is essentially the process of creating the specification, which ultimately forms the foundations for spec-driven development.

What is a spec?

A specification is definitely more than just a product requirements document (PRD). Even simply applying a more structured prompt and more explicit technical constraints can produce better code than a plain PRD.

Technically, a specification should explicitly define the external behavior of the target software — things like input/output mappings, preconditions/postconditions, invariants, constraints, interface types, integration contracts and sequential logic/state machines.

In the past, specifications were often written in highly formalized, machine-readable formats. Today, with the help of LLMs, we can describe them using natural language. Essentially, though, a specification still defines the behavior of the target software; it doesn't just describe business requirements.

What makes a good spec?

Our experiences from behavior-driven development are still valid. This new technology shouldn't actually change that much in this area.

For example, specifications should still use domain-oriented ubiquitous language to describe business intent rather than specific tech-bound implementations. They should also have a clear structure, with a common style to define scenarios using Given/When/Then and should strive for completeness yet conciseness, covering the critical path without enumerating all cases. This has an added benefit for AI-assisted software development in that it can help save tokens.

It's also important that specifications aim for clarity and determinism. While LLMs don't generate deterministic code like traditional code generation, compiling, or automated test execution, clear specifications can still help reduce model hallucinations and produce more robust code.

However, while LLMs primarily excel at handling natural language we shouldn't underestimate the role of structured inputs and outputs. Experience shows that providing the model with semi-structured input prompts or forcing it to output in a structured manner can *significantly* improve reasoning performance and reduce hallucinations. Machine-readable specs, then, remain essential in the LLM era.

Finally, regarding the organization of spec files, many emphasize separating business requirement specs from technical specs. However, in practice, defining the boundary between the two is often unclear.

Spec-driven development in practice

SDD workflows in practice can vary significantly depending on the tools you use. Tools like Amazon Kiro and GitHub Spec-Kit offer some predefined workflows. If you're using a more methodologically neutral AI coding tool, like Cursor or Claude Code, you'll need to find a workflow that suits your needs.

The core of SDD goes beyond vibe coding, separating the design and implementation phases. In the planning phase, requirements are first analyzed using an AI coding agent, which generates design and implementation plans. Typically, these requirements specifications are formalized into different Markdown (.md) files. Reviewing and validating these specifications is usually an iterative process that requires a human in the loop.

Once the specifications are finalized, they're handed to the coding agent to generate the product code, based on the technical requirements (such as architectural style and constraints) you set in Cursor rules/AGENTS.md.

Different people have different opinions on whether specs are just disposable process intermediates or the ultimate truth about software behavior. These differing perspectives also lead to varying workflows for curating and maintaining specs and generating code.

Spec-driven development and context engineering

I often say that prompt engineering optimizes human-LLM interaction, while context engineering optimizes agent-LLM interaction. This is because AI agents typically require more information and larger context windows to complete tasks, posing a significant challenge to LLMs' processing capabilities.

Because coding tasks require a large amount of contextual information, we need to curate that information carefully. Coding agent tools, along with predefined AGENTS.md files, usually provide a good system prompt.

The spec-by-example we typically use in BDD is essentially the few-shot prompt technique [↗](#). Separating requirements analysis and planning from the code implementation phase essentially compresses the context into specs. MCP servers like Context7 can also provide us with real-time documentation information.

Our CodeConcise tool extracts code structure and dependencies from legacy codebases, builds a knowledge graph in vector and graph databases, and can integrate with MCP servers like JIRA and Confluence to support subsequent code generation. Many practices in context engineering can be applied in SDD.

Is spec-driven development just a return to waterfall?

I've heard some people claim this is a return to waterfall — not unreasonably — but I believe this time is different.

The problem with traditional waterfall development is its excessively long feedback cycles. It suffers from a disconnect between software design and implementation, leading to "shadow architecture" and an enormous amount of inefficiency due to the need to maintain code, documentation, and continued testing as requirements change.

The problems we currently encounter with AI coding are different — they stem from the fact that vibe coding is too fast, spontaneous and haphazard. Because it's so easy for AI to generate demonstrable prototypes, many people overlook the importance of good engineering practices, resulting in too much unmaintainable, defective, one-off code.

It's important, then, to bring serious requirements analysis, prudent software design, necessary architectural constraints, and human-in-the-loop governance into the picture. I'd argue that's what spec-driven development helps us to do. It's not creating huge feedback loops like waterfall — it's providing a mechanism for shorter and effective ones than would otherwise be possible with pure vibe coding.

"Spec drift and hallucination are inherently difficult to avoid. We still need highly deterministic CI/CD practices to ensure software quality and safeguard our architectures."

Liu Shangqi

Technology Director, APAC Region, Thoughtworks

The challenges and risks of spec-driven development

As mentioned earlier, there's a lack of consensus on the 'correct' spec-driven development workflow and what exactly a good spec should look like in the context of AI-assisted coding. Although I obviously have my view on what makes a good spec, these are based on my experience — there's not yet a systematic way to evaluate specs as we do with evals, for example.

Code generation from spec to LLMs isn't deterministic; this poses challenges when it comes to upgrades and maintenance. Spec drift and hallucination are inherently difficult to avoid, so we still need highly deterministic CI/CD practices to ensure software quality and safeguard our architectures.

Finally, the question of whether spec or code is the ultimate artifact of software development still needs to be explored. The two lead to entirely different workflows and development practices. Experienced programmers may find that over-formalized specs can cause unnecessary trouble, and slow down change and feedback cycles — just as we encountered in the early stages of waterfall development.

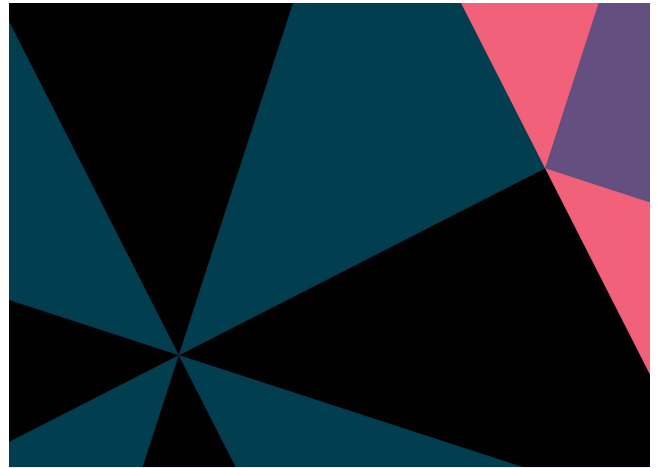
Spec-driven development remains an emerging practice as 2025 draws to a close; we're likely to see even more change in 2026. That means staying on top of industry thinking and the experiments being done is critical. At Thoughtworks we'll continue to experiment and, of course, share our learnings and insights with the rest of the software community.



AI and ML

From vibe coding to context engineering: 2025 in software development

Learn more >



Technology strategy

11 things on Technology Radar Vol.33 that aren't at all related to AI

Learn more >

Technology strategy

Macro trends in the tech industry | November 2025

Learn more >

Start your software engineering transformation

[Learn more](#)

Connect with us



© 2026 Thoughtworks, Inc.